

NAME-CHIRAG

ROLL – 261410202983

SUBJECT CODE - DCA1107

SUBJECT NAME – C PROGRAMMING

PROGRAM NAME - BACHELOR OF COMPUTER APPLICATIONS (BCA)

SEMESTER – 1

Q1.

(a)

A **C program** has a structured template which helps in understanding and executing a program by the compiler. All C programs contain several key entities.

The first section of a C program is the documentation section, where we can place comments which indicates the purpose of the program. The compiler basically ignores these comments; but, they are useful in helping the programmer to understand the code.

After that, we have the header file section, where we can include all the necessary libraries using the `#include` directive. For example, the header file for input and output functions is included by using `#include<stdio.h>`

The `main()` function is the most centralized part of a C program. All programs basically start their execution from the `main()` function. The **main()** function will typically have all variable declarations. After that, we mention all the program statements that complete the goal your program is required to accomplish, such as calculation, output display, or input acceptance.

The final component of a C program is the return statement, which is used to indicate that the program has completed successfully.

Example:

```
#include <stdio.h>
int main()
{
    printf("Hello world");
    return 0;
}
```

The main components of a C program are - comments, headers, main function, variable declarations, program statements, and return statements. If we follow this standardized structure, we can create more effective software that is easily readable and comprehensible, as well as easier to modify later.

(b)

Input and output functions are used to communicate between the user and the computer. In C programming, the input/output functions which are commonly used are **printf()** and **scanf()**. Both these functions are available in the **stdio.h** header file.

The **printf()** function is used to display information on the screen. It helps the program show messages, results, and values of variables to the user.

Example:

```
printf("Welcome to C Programming tutorial");
```

The **scanf()** function is used to accept input from the user. It will help users to enter values while the program is running. The value which the user has entered will get stored in a variable.

Example:

```
int    number;
scanf("%d",
&number);
```

Here, %d is the format specifier for integers, and &num specifies the address of the memory where the value will be stored.

Program Example:

```
#include <stdio.h>
int main()
{
    int number;
    printf("Enter a number:")
    scanf("%d", &number);
    printf("You entered number:
%d", number);
    return 0;
}
```

In this program, **printf()** will display messages and **scanf()** will take input from the user. These functions make C programs interactive and user-friendly.

Q2.

(a)

The **switch-case statement** in C is a decision-making statement which is used to select one option from multiple choices. Instead of using multiple if-else statements, we can use switch-case statement.

Syntax:

```
switch(expression)
{
case      value1:
    statements;
    break;
case      value2:
    statements;
    break;
default:
    statements;
}
```

Working:

The switch statement begins with evaluating the expression within it. Once the expression is evaluated, then the value of the evaluated expression will be matched with the case values and for the matching case, it will execute the case block statements. So, we can say that when a match is found, the break statement will stop the execution of that case, and hence it will exit the switch.

If there are no matching cases, then the default case's statements will be executed.

Example:

```
int  day  =  2;
switch(day)
{
case 1:
    printf("Monday");
    break;
case 2:
    printf("Tuesday");
    break;
default:
    printf("Invalid day");
}
```

In this example, the output will be **Tuesday** because the value of day is 2.

The switch-case statement makes programs easier to read and manage when multiple choices are available.

(b)

Loops are used in C programming to execute a block of code repeatedly until a specified condition becomes false. The three main loops are **for**, **while**, and **do-while**.

1. for Loop

The for loop is used when we know the number of iterations in advance.

Syntax:

```
for(initialization; condition; increment)
{
    statements;
}
```

2. while Loop

The while loop will check the condition before executing the loop body. When we do not know the number of iterations, then while loop is useful.

Syntax:

```
while(condition)
{
    statements;
}
```

3. do-while Loop

The do-while loop executes the loop body first and then checks the condition. Therefore, it executes at least one time even if the condition is false.

Syntax:

```
do
{
    statements;
}
while(condition);
```

Q3.

Operators are special symbols in C. To perform operations on variables and values, we use operators. They can perform calculations, comparisons, logical operations, and assignments. Operators are very important in programming because they allow programmers to process data and get results.

There are various types of operators in C:

1. Arithmetic Operators

These are used to perform mathematical operations.

Operator Meaning

+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus

Program:

```
#include <stdio.h>

int main()
{
    int a = 10, b = 5;

    printf("Addition    =   %d\n",  a   +   b);
    printf("Subtraction  =   %d\n",  a   -   b);
    printf("Multiplication =   %d\n",  a   *   b);
    printf("Division    =   %d\n", a / b);

    return 0;
}
```

2. Relational Operators

When there is a need to compare between two values, we can use relational operators. For example – equal to, greater than, less than, not equal to, etc.

Operator Meaning

==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

Example:

```
int a = 10, b = 20;
printf("%d", a < b);
```

3. Logical Operators

When there is a need to combine two conditions, then we can use logical operators.

Operator Meaning

&&	AND
	OR
!	NOT

Example:

```
int a = 10, b = 20;
if(a < b && b > 0)
{
    printf("Condition True");
}
```

4. Assignment Operators

When we need to assign a value to variables, then we can use assignment operators.

Operator Example

=	a = 5
+=	a += 2
-=	a -= 2
*=	a *= 2
/=	a /= 2

Example:

```
int a = 10;
a += 5;
printf("%d", a);
```

5. Increment and Decrement Operators

When we need to increase or decrease the value of a variable by 1, then we can use increment and decrement operators.

Operator Meaning

++	Increment
--	Decrement

Example:

```
int a = 5;
a++;
printf("%d", a);
```

6. Conditional Operator

The conditional operator (?:) is used as a short form of the if-else statement. If the condition is True, then statement before : will be executed, if the condition is false then statement after : will be executed.

Example:

```
int a = 10, b = 20;
int max = (a > b) ? a : b;
printf("%d", max);
```

Q4.

(a)

In C programming, functions can receive values from the calling function in two ways: **Call by Value** and **Call by Address**. These methods will specify how the data is getting passed to a function.

Call by Value

In call by value method, a copy of the actual argument is passed to the function. If we make any changes to the parameter inside the function, it will not affect the original variable value.

Example:

```
void change(int x)
{
    x = 20;
}
```

If the original value of x is 10, only the copy is modified, so it will not change the original value of x.

Call by Address

In call by address method, the address of the variable is passed to the function using pointers. So, in this case, if we make any changes inside the function, it will directly change the original variable value.

Example:

```
void change(int *x)
{
    *x = 20;
}
```

Here, the original variable value changes because the function accesses its memory location.

Difference

Call by Value

Copy of value is passed

Original value remains unchanged

Uses normal variables

Call by Address

Address of variable is passed

Original value can be changed

Uses pointers

Thus, call by value is safer, while call by address is useful when the function needs to modify the original data.

(b)

Functions are the blocks of code which perform specific tasks. In C, functions are mainly 2 types - **Library Functions** and **User-Defined Functions**.

Library Functions

Library functions are predefined functions which are provided by C libraries. Programmers can use them directly by including the appropriate header file.

Examples:

- printf()
- scanf()
- sqrt()
- strlen()

Example:

```
printf("Hello World");
```

These functions save run time because they are already written and tested.

User-Defined Functions

User-defined functions are created by programmers to perform specific tasks according to program requirements. As per the requirement, we can create custom functions.

Example:

```
int add(int a, int b)
{
    return a + b;
}
```

These functions improve program modularity and code reusability.

Q5.

In C programming, a **string** is a collection of characters. In string, all the characters are stored in a character array and terminated by a null character ('\0'). The C language provides several standard string functions in the **string.h** header file. This helps in performing common string operations easily. These functions save programming time and make string handling more efficient.

The most used string functions are **strlen()**, **strcpy()**, **strcat()**, and **strcmp()**.

1. strlen() Function

The `strlen()` function is used to find the length of a string. It counts the number of characters in a string, excluding the null character.

Syntax:

```
strlen(string_name);
```

Example:

```
#include <stdio.h>
#include <string.h>

int main()
{
    char str[] = "Hello";
    printf("Length = %lu", strlen(str));
    return 0;
}
```

Output:

Length = 5

2. strcpy() Function

The `strcpy()` function is used to copy one string into another string.

Syntax:

```
strcpy(destination, source);
```

Example:

```
#include <stdio.h>
#include <string.h>

int main()
{
```

```

char str1[20], str2[] = "Hello";

strcpy(str1, str2);

printf("%s", str1);
return 0;
}

```

Output:

Hello

3. strcat() Function

The strcat() function is used to join or concatenate two strings. It appends the second string to the end of the first string.

Syntax:

```
strcat(string1, string2);
```

Example:

```

#include <stdio.h>
#include <string.h>

int main()
{
    char str1[20] = "Hello ";
    char str2[] = "World";

    strcat(str1, str2);

    printf("%s", str1);
    return 0;
}

```

Output:

Hello World

4. strcmp() Function

The strcmp() function is used to compare two strings.

Syntax:

```
strcmp(string1, string2);
```

Example:

```
#include <stdio.h>
#include <string.h>

int main()
{
    char str1[] = "Apple";
    char str2[] = "Apple";

    if(strcmp(str1, str2) == 0)
        printf("Strings are equal");
    else
        printf("Strings are not equal");

    return 0;
}
```

Output:

Strings are equal

Q6.

(a)

Recursion is a common means of programming in which a function calls itself repeatedly until a particular stopping criteria has been reached. Recursion is most effective at solving the types of problems that have been divided into smaller, similar subproblems (e.g., calculating the factorial of a number, generating the Fibonacci series, and traversing tree structures).

Some **advantages** to using recursion are:

1. **Simplifies complex problems** – typically easier to write recursion than iteration because the solutions are much shorter.
2. **Reduced code length** – written with fewer lines of code.
3. **Convenience with Trees and Graphs** – recursion works very well for problems that require working with hierarchical data structures.
4. **Improved readability of code** – recursive programs are often easier to read and maintain.

Some **disadvantages** to using recursion are:

1. **Higher memory utilization** – every recursive call takes additional space on the stack.

2. **Decreased speed of execution** – recursive calls incur additional overhead when compared to looping constructs.
3. **Inherent risk of stack overflow** – if the stopping criteria is not present or incorrect, the program could continue to run forever and crash.
4. **Increased difficulty in debugging** – recursive programs can be more difficult to follow and debug.

Example

```
int factorial(int n)
{
    if(n == 0)
        return 1;
    else
        return n * factorial(n - 1);
}
```

Thus, recursion is powerful and useful, but it should be used carefully to avoid excessive memory usage and performance issues.

(b)

A **function prototype** is a declaration of a function that tells the compiler about the function's name, return type, and parameters before the function is used. It helps the compiler check whether the function is being called correctly.

Syntax

```
return_type function_name(parameter_list);
```

Example

```
int add(int a,int b)
{
    return a+b;
}
int main ()
{
    Int result;
    result  =  add(5,3);
    return 0;
}
```

In this example, `int add(int, int);` is the function prototype.

The significance of function prototypes -

1. Function prototypes give the compiler information about the function before calling it.

2. Function prototypes allow the compiler to identify errors pertaining to argument number and types.
3. Function prototypes allow for greater program organization since functions can now be defined after `main()`.
4. Function prototypes assist in enhancing the readability of larger programs; they help make them more manageably understandable and maintainable.
5. Function prototypes can be used to support modular programming as different functions can be developed independently.